Verified by Construction: Machine-Checked Proofs for Hand-Written Transformer Circuits and Their Composition

Xintian Huang
Independent Researcher
xintian_huang@outlook.com

Abstract*—When transformer weights are written by hand or compiled in rather than trained, an auditor needs to know what they compute on every input. Sampled evaluation cannot tell: a hand-written multiplication transformer with 100% sampled accuracy answers `89984*48898=` wrongly. We give machine-checked Lean 4 proofs that follow the mechanism the weights were written for. First, we prove seven hand-written weight-space programs correct on every input of their task, with up to about 1.3M parameters, up to $26^9 \approx 5.4 \times 10^{12}$ prompts, and real LayerNorm. Second, for circuits appended to the residual stream of a trained host, we prove a grafting theorem for any host in our formalised pre-LayerNorm architecture with positive width and LayerNorm ε, on prompts within its context length. For circuits whose features are zero-sum with constant energy, LayerNorm compensation makes an idle circuit with positive LayerNorm ε leave the host's logits exactly unchanged. On prompts whose host variance is at most a stated σ^2, a circuit with zero-sum writes moves the logits at a position by at most an explicit bound if, for every variance profile up to σ^2, its readout there vanishes for every token and its final energy stays in a band $[E, E + \Delta]$ starting at its features' energy E; and its token wins if, for every such profile, its energy is at least that of its features and its margin beats the host gap of every other token. The circuit's obligations use the host's weights only through its unembedding, output bias and final LayerNorm, and the bound also uses its variance, so the proofs never load host weights. Third, we instantiate the theorem as a multi-token refusal rule, a key gate with a counting bound on triggering by uniformly random prompts, and a watermark with false-positive rate at most $e^{-m/8}$ for a uniformly random secret code independent of the reading. Grafted into OPT-6.7B, the proved refusal circuit fires on 2480 of 2480 trigger positions in float32 and float16, and on the evaluation windows its float64 off-trigger deviation stays below 1.1% of each token's proved bound. The theorems cover the exact real-valued weights. For the circuits we instantiate we prove, under their hypotheses, that every circuit LayerNorm radicand is positive, so their guarantees hold for ordinary real-arithmetic execution with heads of positive dimension; floating-point storage and execution are measured, not proved.

1. Introduction

A model that scores 100% on its evaluation can still be wrong. One of the hand-written multiplication transformers we study reached 100% on its sampled evaluation (200 examples by default). Yet both of its final snapshots answer `89984*48898=` incorrectly, in float32 and in float64; our own 20,000-sample check found this single failure (section 3). Samples certify the samples.

This matters more as weights become a medium for writing behaviour directly. LLM agents now write network weights directly, without training [1], [2], including small transformers for algorithmic tasks judged by sampled accuracy [2]. RASP programs [3] are compiled into weights by Tracr [4]. And circuits are appended to the residual stream of a trained host to add capabilities such as cryptographically gated backdoors [5]. In each case the question an auditor asks is the same: what do these weights compute, on every input, and what do they leave unchanged?

The auditor's position. Consider a provider who appends a safety circuit, such as a refusal rule, to a deployed model, and an auditor who must accept or reject the edited model without trusting the provider's tests. A test set cannot settle this: it says nothing about inputs outside the sample, as the multiplication example shows. With our construction, as in proof-carrying code [6], the provider hands over the weights, a Lean theorem and its proof, and each part of the guarantee has a definite checker:

- *Kernel-checked:* the theorem, for the exact real-valued graft of any admissible host.
- *Checked on the weights by trusted code:* the zero input columns and mean output rows of the host matrices, and the circuit weights and circuit LayerNorms against the Lean definitions, both up to the rounding of the stored values, and the theorem's parameter inequalities.
- *A runtime hypothesis:* the host variance at each position is at most σ^2; the deployed model can monitor it by computing one variance per position.
- *Measured, not proved:* that floating point stays close to the exact construction. The theorems are about real-valued weights, the stored tensors round them (the LayerNorm compensation factor $\sqrt{D/(D+k)}$ is in general irrational), and execution rounds again.

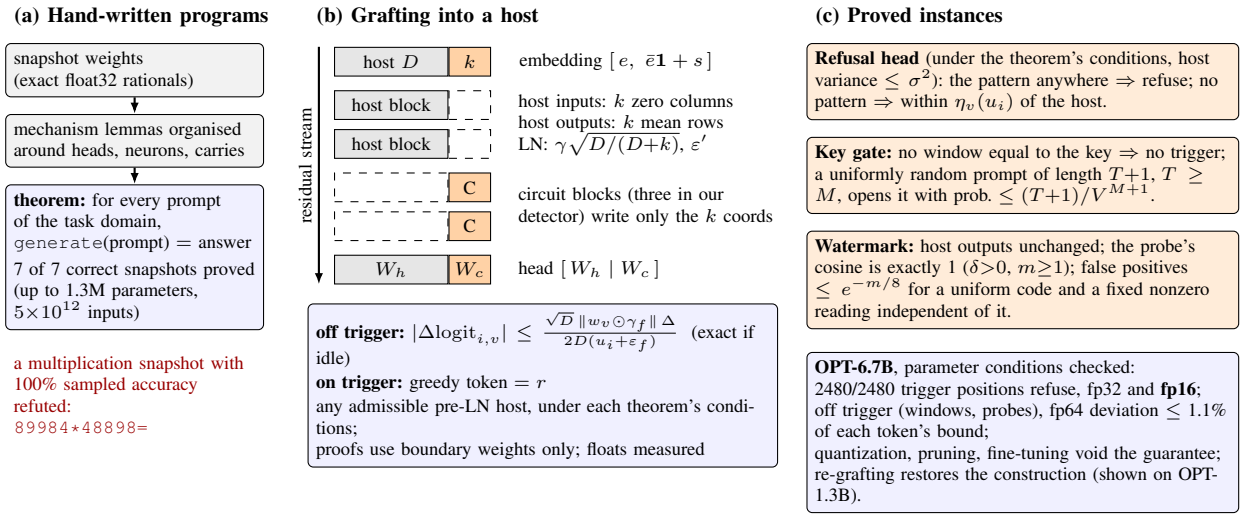


Figure 1. Overview. (a) Hand-written transformer programs are proved correct on every input, along their mechanism. (b) A circuit with zero-sum, constant-energy features grafted onto k new residual coordinates with LayerNorm compensation preserves any admissible host ($D > 0$, positive LayerNorm ε), under each theorem's hypotheses: exactly when idle with positive circuit LayerNorm ε (theorem 2); and, for host variances up to σ^2, within a position-wise bound when, for every variance profile up to σ^2, its readout vanishes and its energy stays in $[E, E + \Delta]$ (theorem 4). On trigger its token wins when, for every such profile, its energy is at least E and its margin beats the host gap of every other token (theorem 3). (c) Three safety instances proved on top of the graft, and the evaluation of the refusal head on OPT-6.7B.

The checks on the weights are scripts that we ran on our evaluated configurations (section 6), not a packaged audit tool.

Thesis. Weights written for a mechanism can be verified along that mechanism. The circuits we study were written for explicit mechanisms, and their correctness proofs decompose the same way: lemmas organised around heads, neurons and carries. For circuits grafted into a trained host, our construction extends function-preserving width expansion [7], which fills new residual coordinates with the host's mean and rescales every LayerNorm, from empty coordinates to coordinates that carry a working circuit, whose energy the LayerNorm compensation absorbs. We test the thesis at three scales, all in Lean 4 with Mathlib [8], [9], with every theorem depending only on the kernel's three standard axioms.

Contributions. Figure 1 summarises them.

1) **A verified benchmark of weight-space programs** (section 3). Of the ten algorithmic tasks of a weight-space-programming study [2], we prove seven snapshots correct on every input of their generator (input spaces up to $26^9 \approx 5 \times 10^{12}$; models up to about 1.3M parameters; real LayerNorm in two of them), and we refute one snapshot with 100% sampled accuracy by a counterexample. Proof-line counts are not proportional to parameter counts: addition has 1,300,512 parameters and 1,699 proof lines, sort has 4,364 parameters and 990.
2) **A grafting theorem for pre-LayerNorm transformers** (section 4). For any host in the semantics of section 2 with width $D > 0$ and positive LayerNorm ε, we append k residual coordinates and compensate every LayerNorm. For circuits with zero-sum, constant-energy features and prompts within the context length we prove that an idle circuit with positive circuit LayerNorm ε leaves every logit exactly unchanged; that a circuit with zero-sum writes whose readout vanishes and whose final energy stays in $[E, E + \Delta]$, for every variance profile up to σ^2, changes each logit by at most an explicit position-wise bound; and that a circuit with zero-sum writes whose final energy is at least E and whose margin beats the host gap of every other token, for every such profile, makes its token win. The circuit's obligations use the host's weights only through its unembedding, output bias and final LayerNorm (the bound also uses its final-residual variance), the last two results assume host variance at most σ^2, and the proofs never load host weights. Our multi-token detector writes values that depend on the host's variance, so it uses the approximate result.
3) **Three proved instances, evaluated up to OPT-6.7B** (sections 5 to 7): a refusal head for a multi-token pattern; key gating with a counting bound $(T+1)/V^{M+1}$ on triggering by a uniformly random prompt of length $T+1 \geq M+1$; and a watermark with two white-box detectors and a false-positive bound $e^{-m/8}$ for a uniformly random code and a reading fixed independently of it. For every circuit we use we prove, under its theorem's hypotheses, that every LayerNorm radicand stays positive. Grafted into OPT, the refusal circuit's parameters meet the theorem's conditions, also rechecked in 60-digit arithmetic; on OPT-125m and OPT-1.3B (float64) its weights match an independent

transcription of the Lean definitions; the variance hypothesis holds on every evaluation window, near-miss prompt and inserted-trigger prompt, and an adversarial search reaches 97.8% of σ^2. In the evaluated low-gain grafts, every tested trigger position refuses in float16 and under the tested int4 quantizers. Quantization, pruning and fine-tuning void the guarantees, and re-grafting restores the construction.

Stance. Key gating uses the trigger mechanism of grafted backdoors [5]: a fixed token sequence switches on a circuit appended to the host. Under the hypotheses of `key_main`, our proofs state what the circuit does on every prompt within the context length, and the key is stored in plain text, so anyone holding the weights can read it (section 7).

2. Setting

Semantics. We formalise a decoder-only, pre-LayerNorm transformer over $\mathbb{R}$ (`GArch`). A host has width D, vocabulary V, context length L, token and learned position embeddings, a list of blocks, a final LayerNorm $(\gamma_f, \beta_f, \varepsilon_f)$ and an unembedding W_h. Each block applies $x \mapsto x + \mathrm{Attn}(\mathrm{LN}_1\, x)$ and then $x \mapsto x + \mathrm{MLP}(\mathrm{LN}_2\, x)$.

- LayerNorm is $\mathrm{LN}(y) = \gamma \odot (y - \bar{y})/\sqrt{\mathrm{var}(y) + \varepsilon} + \beta$ with the population variance.
- Attention is causal softmax attention with scores $q \cdot k/\sqrt{d_q}$ and biases on every projection.
- The MLP activation is arbitrary (GELU in GPT-2 [10], ReLU in OPT [11]).
- Predictions use $\arg\max$ with first-index tie-breaking, as in PyTorch.

Phase-1 models embed into this semantics with equal logits (`Model.toG_logits`). OPT models match it exactly at every size except 350M. We check this against the OPT implementation in the Transformers library [12] (configurations in `evidence/opt_configs.txt`): it uses pre-LayerNorm blocks, learned positions at row offset 2, and a tied unembedding.

Trusted base. Every theorem is checked by the Lean 4 kernel and depends only on `propext`, `Classical.choice` and `Quot.sound`. No theorem uses `sorry` or `native_decide`. Statements are over $\mathbb{R}$. Float behaviour is a separate layer. Only the parity model has float theorems (`Float.lean`, `Binary32.lean`): every operation is rounded by an operator with $|\mathrm{rnd}(x) - x| \le 10^{-7}|x| + 10^{-12}$, sums are taken sequentially left to right, and exp has relative error at most 10^{-6} on $[-1, 1]$. The instance is binary32 round-to-nearest-even on the real grid (without an overflow bound; all intermediates stay below 400), with either such an exp or a correctly rounded one. The link to executable float operations and GPU kernels (other summation orders, FMA) is not proved. Everywhere else we measure float behaviour and do not prove it.

From weights to Lean. An exporter checks the snapshot's SHA-256 and writes every float32 parameter as its exact rational value. For the six snapshots exported to the generic semantics, it then compares a float64 implementation of that semantics with the snapshot's own float64 PyTorch forward pass (with LayerNorm ε set to its float32 value), on every input where the input space allows (boolean circuit, sort, linear interpolation) and on 20,000 sampled inputs otherwise (`evidence/export_provenance.txt`). Parity has its own exporter, which compares the snapshot's float32 and float64 outputs on all 2,046 inputs. That the Lean constants equal the deployed parameters rests on this exporter, not on a proof.

Who asks what. An auditor holds the weights and wants to know what they compute. Our theorems answer that question for every input of the stated domain; modified weights and adversaries who rewrite them are treated in section 7.

3. Verified Weight-Space Programs

We take the ten algorithmic tasks of a recent weight-space-programming study [2], in which, as that study describes, an agent writes the weights by hand and evaluates each snapshot on sampled inputs (200 by default). For seven selected snapshots, the final versions except as noted for word reversal, we state, in Lean, that greedy generation produces the task's answer on every prompt of the task generator, and prove it along the snapshot's mechanism; the multiplication snapshot is refuted by a counterexample instead. Table 1 gives the specifications. Each is a Lean theorem over the whole input domain. For addition, for example, it reads

```
theorem add_correct (A B : Num5) :
 addModel.generate (prompt A B) 6
        = some (answer A B),
```

where `prompt` and `answer` are the token sequences of the generator's string format and `generate` is greedy decoding with first-index $\arg\max$. Parity and the boolean circuit predict a single token and are stated for that prediction.

Outcome. Table 2 lists the ten tasks.

- Seven snapshots are proved correct on every input.
- Both final multiplication snapshots scored 100% on the study's evaluation (200 examples by default). Our 20,000-sample check of the smaller one found one failure, `89984*48898=`. The product is 4400037632; both snapshots generate 4300037632 in float32 and 4390037632 in float64 (`evidence/mult_counterexample.txt`). The second digit is wrong in both precisions, with top-two margins of 1.31 (float32) and 1.35 (float64); in float64 the third digit also flips, at a margin of 0.03.
- Decimal-to-binary is a Python loop rather than a weight program; we proved its transcription as a program and do not count it.

TABLE 1. SPECIFICATIONS, AS THE LEAN THEOREMS STATE THEM OVER TOKEN SEQUENCES (THE GENERATORS' STRING FORMATS). EVERY PROMPT IN THE DOMAIN IS COVERED.

Task	Prompt → answer	Domain
parity	bits left-padded to 10 with _, then = → parity of the number of 1s	1 to 10 bits
boolean circuit	$b_0 o_0 b_1 o_1 b_2 o_2 b_3 o_3 b_4$= → the chain evaluated strictly left to right	b_i bits, $o_i \in \{\wedge, \vee, \oplus\}$
digit counting	$d_0 \dots d_9\ q$= → the number of i with $d_i = q$, as two digits	all digit strings
word reversal	w_1_w_2_w_3= → w_3_w_2_w_1	w_i three letters from 26
sort	$a_0 \dots a_4$= → the digits in ascending order	all digit strings
linear interpolation	$x_1, y_1; x_2, y_2; x_q$= with $y = (m{+}1)x{+}b \to y(x_q)$, two digits	digits $x_1 \neq x_2$, x_q, b; $m < 5$
addition	$A{+}B$=, five digits each → $A{+}B$, six digits zero-padded	all digit strings

- GCD is a 998,001-unit lookup table stored in buffers; we report the finding and do not prove it.

Every correct in-scope snapshot we attempted was proved: 7 of 7.

A worked example: addition. The proof follows the snapshot's three blocks.

1) *Weight structure.* One lemma per exported matrix states its hand-written form, for example that each of block 0's 1,024 MLP rows is an AND unit of two digit slots or zero.
2) *Gather.* Ten block-0 heads copy the ten operand digits into slots; softmax leaks at most $\lambda \leq 1/8000$ of the mass elsewhere.
3) *Generate and propagate.* 500 AND units $\mathrm{relu}(A_i[a] + B_i[b] - 1)$ sum to g_i $(a + b \geq 10)$ and p_i $(a + b = 9)$, each within 2λ of its 0/1 value.
4) *Carry.* Block 1's 15 AND units cover the disjoint carry paths $p_k \wedge \cdots \wedge p_{j-1} \wedge g_j$; their sum is within 30λ of the carry.
5) *Emit.* At each answer position one block-2 unit, the one for the true digits and carry, fires near 1 and writes the answer digit, while all other units together contribute at most $1/2$, so the first-index $\arg\max$ is the correct digit.

Induction over the six generation steps, each extending the prompt by the digits already emitted, gives `add_correct`.

Proof size. Two reviews, recorded in `audit/`, each wrote a specification from the Python sources and then compared it with the seven Lean theorem statements; neither found a mismatch. Proof-line counts are not proportional to parameter counts.

- Addition, with 1.3M parameters, needs 1,699 hand-written lines.
- Sort, with 4,364 parameters, needs 990.

What did not scale was the per-neuron weight-structure step. For the 1,000- and 1,024-neuron layers of linear interpolation and addition we used column-major export and `fin_cases` over each row, and addition's weight-structure lemmas were split across modules to fit in memory (section B).

What the audit found. A larger sample, source inspection and the proofs together surfaced four kinds of issue:

1) *A snapshot with 100% sampled accuracy can be wrong* (multiplication).
2) *Correctness can hinge on things outside the weights.*
   - Digit counting's tens digit is an exact all-zero tie for counts below ten, decided by PyTorch's first-index $\arg\max$.
   - Word reversal's weights depend on an environment variable that sets an attention scale (`AR_SCALE`, default 50, which also sets the head scale by default). The study's final recorded run used scale 200; we prove the default configuration, scale 50.
3) *"Zero parameters" can be bookkeeping.* Decimal-to-binary is a Python loop; GCD keeps 7.4M buffer elements, values and sparse indices, outside its parameters.
4) *Correctness can rest on quantitative bounds.*
   - Sort's head puts only 92–97% of its attention mass on the input digits (`evidence/sort_analysis.txt`), so the proof bounds the leaked mass and the clamp outputs by interval arithmetic.
   - Addition's AND units $\mathrm{relu}(a + b - 1)$ have no margin: on exact 0/1 inputs they output exactly 0 or 1. The proof therefore rests on a gather-leak bound $\lambda \leq 1/8000$.

   The proofs therefore need quantitative leak and interval bounds rather than "ideal circuit" reasoning.

4. Grafting Circuits into Pre-LayerNorm Hosts

4.1. Construction

Let M be a host of width D and C a circuit on k new residual coordinates. C consists of token features $s_{\mathrm{tok}}(v) \in \mathbb{R}^k$ and position features $s_{\mathrm{pos}}(p) \in \mathbb{R}^k$, whose sum $s(v, p) = s_{\mathrm{tok}}(v) + s_{\mathrm{pos}}(p)$ is the feature of token v at position p; a list of circuit blocks; final-LayerNorm gains and biases $\gamma_f^c, \beta_f^c \in \mathbb{R}^k$ for the new coordinates; and readout rows $W_c \in \mathbb{R}^{V \times k}$. The graft $\mathrm{graft}(M, C)$ has width $D + k$ (`Construct.lean`):

- *Embeddings.* A token row e becomes $[\, e,\ \bar{e}\mathbf{1} + s_{\mathrm{tok}}(v)\,]$ and a position row $[\, e,\ \bar{e}\mathbf{1} + s_{\mathrm{pos}}(p)\,]$: the new coordinates hold the host row's mean plus the circuit feature.
- *Host blocks.* Input matrices get k zero columns, so the host never reads the circuit. Output matrices and

TABLE 2. OUTCOME PER TASK. PARAMETERS ARE TOTAL (NONZERO). "STEPS" IS THE NUMBER OF GENERATED ANSWER TOKENS. ALL PROOFS ARE OVER $\mathbb{R}$ WITH THE FLOAT32 PARAMETER VALUES THAT THE EXPORTER WRITES (SECTION 2). INPUT SPACES ARE THOSE OF THE STUDY'S TASK GENERATORS. SOURCES: `WEIGHTS/PHASE1_SURVEY.JSON`, `PROOF_RESULTS.CSV`.

Task	Params	Architecture	Steps	Input space	Outcome
parity (up to 10 bits)	174 (35)	1 layer, no LN	1	2,046	proved (also float: binary32 rounding model)
boolean circuit (5 bits)	5,592 (223)	attn+MLP, 3 MLP-only blocks	1	2,592	proved
digit counting	4,054 (303)	2 layers, 2 heads	2	10^{11}	proved
word reversal (3×3, scale 50)	14,893 (357)	1 layer, real LN	11	26^9	proved
sort (five digits)	4,364 (384)	1 layer, real LN $\times 3$	5	10^5	proved
linear interpolation	709,104 (10,203)	2 layers, 8 heads	2	45,000	proved
addition (five digits)	1,300,512 (7,449)	3 layers, 10 heads	6	10^{10}	proved
multiplication (five digits)	22,166,060 (67,813)	11 layers, real LN	10	10^{10}	**refuted** (`89984*48898=`)
decimal to binary	0 (253 buffer elements)	Python loop over digits	8	256	out of scope; proved as a program
GCD (three digits)	0 (7.4M buffer elements)	lookup MLP in buffers	3	998,001	not attempted (finding only)

biases get k copies of their mean row, so every write keeps the new coordinates equal to the host's running mean plus the circuit's own state.

- *LayerNorms.* Every host LayerNorm is compensated: $\gamma \mapsto \gamma\sqrt{D/(D+k)}$ and $\varepsilon \mapsto \varepsilon' = (D\varepsilon - E)/(D+k)$, where E is the circuit features' energy.
- *Circuit blocks.* Appended after the host blocks; they read and write only the new coordinates. Their LayerNorms run over all $D+k$ coordinates with $\gamma = \beta = 0$ on the host coordinates and the same ε compensation.
- *Final LayerNorm.* Gains $[\,\gamma_f\sqrt{D/(D+k)},\ \gamma_f^c\,]$, biases $[\,\beta_f,\ \beta_f^c\,]$ and $\varepsilon'_f = (D\varepsilon_f - E)/(D+k)$.
- *Head.* The unembedding becomes $[\,W_h \mid W_c\,]$. We write w_v for the host's unembedding row of token v and b_v for its output bias.

Circuit features are *zero-sum* ($\sum_j s(v,p)_j = 0$) and have *constant energy* ($\sum_j s(v,p)_j^2 = E$) for every token v and position p.

Standing assumptions. A host is *admissible* if its width is $D > 0$ and every host LayerNorm, including the final one, has $\varepsilon > 0$. Every result below assumes an admissible host and prompts no longer than the context length L (and nonempty prompts wherever a next token is predicted); the Lean statements carry these as explicit hypotheses. As in Lean, all host blocks share one shape (MLP width, number of heads, head dimensions). Our implementation of the evaluated OPT hosts uses $\varepsilon = 10^{-5}$ in every LayerNorm.

Total and ordinary semantics. Lean's square root and division are total ($\sqrt{x} = 0$ for $x < 0$, $x/0 = 0$), and the theorems are stated in this total semantics. For ordinary real-arithmetic execution we also require every attention head, of the host and of the circuit, to have query dimension $d_q \geq 1$ (true of the evaluated OPT models and of all our circuits; Lean's host type allows $d_q = 0$), and we treat activations as ordinary real functions (ReLU in OPT). Then the attention scale $1/\sqrt{d_q}$ and the softmax denominators are positive, and the only remaining operations that are undefined in ordinary real arithmetic are the square root and division inside LayerNorm. Such a run therefore coincides with ordinary real-arithmetic execution whenever every LayerNorm radicand it meets is positive. For theorem 2 and the exact case of lemma 1 this always holds. For theorems 3 and 4 it does not follow from their hypotheses: zero-sum writes can take the circuit's energy below E between blocks (for instance, one block erases the features and the next restores them), making an intermediate radicand negative while the final energy is again E. Read as statements about ordinary real-arithmetic execution, theorems 3 and 4 therefore carry the additional hypothesis that every circuit LayerNorm radicand on the prompt is positive (`RadPos`). We prove this hypothesis for every circuit we use (`Radicands.lean`): `pattern_radicands_pos` for the pattern detector and the key gate, under the hypotheses of theorem 5, and `refusal_radicands_pos` for the single-token refusal head; the watermark has no circuit blocks and is covered by theorem 2. Floating point is treated in section 7.

Lemma 1 (LayerNorm of the graft, `ln_graft_gen`)**.** *Let $D > 0$, let $y \in \mathbb{R}^D$ be the host state and $s \in \mathbb{R}^k$ a zero-sum circuit state. The compensated LayerNorm of $[\,y,\ \bar{y}\mathbf{1} + s\,]$ has host coordinates*

$$\frac{y - \bar{y}}{\sqrt{g}}\,\gamma\sqrt{\tfrac{D}{D+k}} + \beta, \qquad g = \frac{D(\operatorname{var} y + \varepsilon) + \|s\|^2 - E}{D+k},$$

and circuit coordinates $\gamma_c\, s/\sqrt{g} + \beta_c$ (for $g > 0$; Lean's square root and division are total, so the identity holds as stated in Lean for every g). If $\|s\|^2 = E$ and $\varepsilon > 0$, the host coordinates equal $\mathrm{LN}(y)$ *exactly (`ln_graft`).*

The lemma is the trap that some hand-written programs avoid by deleting LayerNorm: a naive graft (zero padding, unchanged LayerNorms) can change the host's normalisation. On OPT-1.3B it moves host logits by up to 9.9 and changes 10.1% of greedy tokens (section 6).

4.2. Three theorems

Write $u_t = \operatorname{var}(y_t)$ for the host's final-residual variance at position t; it depends only on the host and the prompt.

Theorem 2 (Exact, idle circuit; `graft_offtrigger`)**.** *Let the host be admissible, every circuit LayerNorm have $\varepsilon > 0$, and the circuit features be zero-sum with constant energy E. Suppose that on a prompt the circuit is* idle *(`Circuit.Idle`): evaluated on the initial circuit state,*

every block's attention writes nothing for every positive per-position LayerNorm scale, every block's MLP writes nothing for every positive scale, and the readout is zero for every positive scale. Then every grafted logit at every position equals the host's.

Theorem 3 (On trigger; `graft_ontrigger`)**.** *Let the host be admissible, the circuit features be zero-sum with energy E, and every circuit block write zero-sum updates. Fix a prompt and a position i. On this prompt the grafted residual equals the host residual joined with a k-dimensional circuit state that depends on the host only through the variances u (`host_blocks`, `circuit_blocks_run`). Write* $\mathrm{out}(u)$ *for the circuit's output at i after the final (compensated) LayerNorm under the profile u. Suppose that for every profile u with $0 \le u_t \le \sigma^2$ the circuit's final state at i has energy at least E and*

$$\textstyle\sum_j (W_c[r] - W_c[t])_j \,\mathrm{out}_j(u) > \mathrm{gap}(r,t) \quad \textit{for all } t \neq r,$$

$$\mathrm{gap}(r,t) = \sqrt{D}\,\big\|(w_t - w_r)\odot\gamma_f\big\| + (w_t - w_r)\cdot\beta_f + (b_t - b_r).$$

Then, if the prompt's host variances are at most σ^2, token r has the strictly largest grafted logit at position i (in the total semantics; for ordinary real-arithmetic execution, if moreover every head has $d_q \ge 1$ and every circuit LayerNorm radicand on the prompt is positive).

The gap uses only the host's boundary weights (unembedding, output bias and final LayerNorm), by Cauchy–Schwarz; no host weight is ever loaded into a proof. `graft_main` packages theorems 2 and 3 with a circuit *contract* (`Circuit.Contract`): zero-sum, constant-energy features; positive circuit LayerNorm ε; zero-sum writes; idle on every non-trigger prompt; and on every trigger prompt, at the last position and for every variance profile up to σ^2, final energy at least E and a margin above the gap. Its corollary `graft_preserves_spec` transfers every next-token property of the host on non-trigger prompts to the graft. A concrete single-token refusal head meets the contract for every admissible host, given $\sigma^2 \ge 0$ and a readout gain L_g above an explicit threshold computed from σ^2, the width D, the final LayerNorm ε_f, the context length L and an upper bound $G \ge 0$ on the host gaps (`refusal_main`), so the contract is satisfiable.

Why our pattern detector uses an approximation. Theorem 2 needs an idle circuit. Our multi-token detector is not idle: it stores partial matches by writing. The exact route of lemma 1 needs the circuit's energy to be E wherever the lemma is applied (a sufficient condition for exact host normalisation, not a necessary one for equal logits). Some writes keep it there: our bit flips do. But in our detector the attention copy values a_d depend on the host's variance profile, and their writes into initially zero copy coordinates add $2\sum_d a_d^2$ to the circuit's energy, so that energy varies from host to host. We take the approximate route instead. Exactness is not ruled out for every multi-token trigger: two attention heads whose writes cancel exactly on non-trigger prompts can detect a two-token conjunction while idle on every other prompt (section C). Our formalized multi-token detector uses the approximate off-trigger guarantee.

Theorem 4 (Position-wise approximation; `graft_offtrigger_approx_at`)**.** *Let the host be admissible, the circuit features be zero-sum with energy E, and every circuit block write zero-sum updates. Fix a prompt whose host variances are at most σ^2 and a position i. Suppose that for every variance profile in $[0,\sigma^2]$ the circuit's final state at i has energy in $[E, E+\Delta]$ and its readout at i vanishes. Then for every token v,*

$$\big|\,\mathrm{logit}^{\mathrm{graft}}_{i,v} - \mathrm{logit}^{M}_{i,v}\big| \;\le\; \eta_v(u_i) = \frac{\sqrt{D}\,\|w_v\odot\gamma_f\|\,\Delta}{2D\,(u_i+\varepsilon_f)}$$

(in the total semantics; for ordinary real-arithmetic execution, if moreover every head has $d_q \ge 1$ and every circuit LayerNorm radicand on the prompt is positive).

The bound uses the host's own variance at i and needs no lower bound on it. At $u_i = 0$ it is the uniform bound $\sqrt{D}\|w_v \odot \gamma_f\|\Delta/(2D\varepsilon_f)$ (`graft_offtrigger_approx`); at a variance u_i it is smaller by the factor $\varepsilon_f/(u_i+\varepsilon_f)$, which for the OPT hosts we evaluate ($\varepsilon_f = 10^{-5}$, measured $u_i \ge 0.17$) is below 10^{-4}. A companion lemma (`argmax_stable`) keeps the host's greedy token r wherever logit_r exceeds every other logit_v by more than $\eta_v + \eta_r$.

Proof idea. All three results rest on one invariant: the grafted residual is $[\,y,\ \bar{y}\mathbf{1}+s\,]$, with y the host's own residual and s a zero-sum circuit state. The embeddings establish it; a host block preserves it because its input matrices ignore the new coordinates and its mean output rows add the mean of its host write to every new coordinate; a circuit block preserves it because it writes zero-sum updates to the new coordinates only. By lemma 1, every compensated LayerNorm then returns $\alpha(\mathrm{LN}(y) - \beta) + \beta$ on the host coordinates, with $\alpha = \sqrt{D(u+\varepsilon)/(D(u+\varepsilon)+q)}$, where u is the host variance and $q = \|s\|^2 - E$. For theorem 2, an idle circuit never changes s, so $q = 0$, $\alpha = 1$, and with a zero readout every logit equals the host's. The circuit LayerNorms see the host only through the scale $1/\sqrt{g}$, which is why idleness and the trigger conditions are quantified over all positive scales or all variance profiles up to σ^2. For theorem 4, the readout vanishes, so the logit difference is $(\alpha - 1)\, w_v \cdot ((y - \bar{y}) \odot \gamma_f)/\sqrt{u+\varepsilon_f}$. Cauchy–Schwarz gives $|w_v \cdot ((y-\bar{y})\odot\gamma_f)|/\sqrt{u+\varepsilon_f} \le \sqrt{D}\,\|w_v \odot \gamma_f\|$, and $0 \le 1 - \alpha \le q/(2D(u+\varepsilon_f))$ with $q \in [0,\Delta]$, which yields η_v. For theorem 3, $q \ge 0$ gives $\alpha \le 1$, the same Cauchy–Schwarz step bounds the host's contribution to $\mathrm{logit}_t - \mathrm{logit}_r$ by $\mathrm{gap}(r,t)$, and the circuit's margin exceeds it.

5. Three Proved Safety Instances

5.1. Refusal on a multi-token pattern

Given a pattern P of $M+1$ tokens and a refusal token r, under the hypotheses of theorem 5 the detector fires at

position i (its flag becomes positive) if P occurs contiguously and ends at or before i (`PatternCirc.lean`); the theorem below states the consequence for the next token. It has three circuit blocks; all circuit LayerNorms have $\gamma = 1$, $\beta = 0$, $\varepsilon = 1$ before graft compensation.

Every quantity is held in a *mirrored* pair $(x, -x)$ or a quad $\delta(b, -b, 1-b, -(1-b))$. Writes are therefore zero-sum, and reads $(x_0 - x_1)/2$ cancel the mean-tracked part exactly over $\mathbb{R}$. In floating point both coordinates receive the same mean, and one diagnostic measures the effect: at the match position of each trigger prompt, the smallest copied bit (exactly 1 over $\mathbb{R}$) is at least 0.9999991 in the unmodified float32 OPT grafts and 0.99975 in the low-gain float16 grafts of OPT-1.3B and OPT-6.7B (low gain: large copy and flag gains, hence small compensating weights; section 6.2).

- *Block 1, copy and compare.*
  - Head d uses ± 1 binary position codes and attends to position $\max(i - (d{+}1), 0)$; its score scale $c_s = c_q c_k$ is split between query and key.
  - It copies $g_v \cdot [\text{token} = P_{M-1-d}]$, scaled by δ and the circuit LayerNorm factor, with leakage bounded by the attention's sharpness.
  - A clamped ReLU pair $\text{clamp}(z) = \text{relu}(z) - \text{relu}(z-1)$, applied to the thresholded and amplified copy $z = W_a \kappa_2 (a - \tau g_v \delta)$, where a is the copied value and κ_2 the next circuit LayerNorm factor, turns it into an *exact* bit.
- *Block 2, AND.* A clamp of $W_b\, \delta\, \kappa_2 \big(\frac{1}{2} - \sum(1-b)\big)$, where κ_2 is the circuit LayerNorm factor, over the complements of the $M{+}2$ bits: the current token, "$i \geq M$", and the M copied bits. The complements sum to 0 iff all bits are 1 and to at least 1 otherwise, so the threshold sits mid-gap and the AND is exact.
- *Block 3, flag.* A head attends to the positions whose match bit is 1 (query $f_q \cdot \text{const}$, key $f_k \cdot \text{match}$) and writes A times the match read into a flag pair.
- *Readout.* $\frac{L_g}{2}(\text{flag}_0 - \text{flag}_1)$ into token r, applied to the final-LayerNorm output: a raw flag pair $(f, -f)$ contributes $L_g f / \sqrt{g_{\text{fin}}}$, where g_{fin} is the final LayerNorm's radicand.

Theorem 5 (`pattern_main`)**.** *Let $M \geq 1$, let the host be admissible with final-residual variance at most σ^2 on a prompt of length at most L, and let the parameters meet the conditions* Good *(positivity and sign conditions, $L \leq 2^n$, h1, h3, h4, h6; section D) and, with $G_0 \geq 0$, the readout condition*

$$G_0\, \Gamma < L_g\, A\, \delta\, \kappa_{2,\text{lo}}\, (1 - \lambda_3), \quad \text{gap}(r, t) \leq G_0 \ (t \neq r),$$

- *if P occurs nowhere, every grafted logit at position i is within $\eta_v(u_i)$ (theorem 4) of the host's, with $\Delta = 2M(g_v \delta \kappa_{\text{hi}})^2$;*
- *if P occurs, the grafted greedy next token is r.*

Under the same hypotheses every LayerNorm radicand of the run is positive (`pattern_radicands_pos`), so both conclusions also hold for ordinary real-arithmetic execution of any host whose attention heads have $d_q \geq 1$.

The conditions are explicit inequalities in D, L, σ^2 and the parameters (section D):

- h1 bounds the copy heads' leakage $\lambda = \exp(-2 c_s \delta^2 \kappa_{\text{lo}}^2 / \sqrt{n+1})$;
- h3 and h4 make the bit and AND clamps saturate;
- h6 requires the flag head's leakage $\lambda_3 = L \exp(-f_q f_k \delta^2 \kappa_{2,\text{lo}}^2 / \sqrt{n+1})$ to be below 1.

Partial matches move the circuit's energy by at most Δ but never its readout: off trigger the match bits are exactly zero, so the flag is exactly zero. That is precisely the situation theorem 4 covers.

The gain is a parameter. Any parameters meeting the conditions are covered by the same theorem.

- *Low gain* ($g_v = 0.3$, $A = 100$): the largest circuit weight on OPT hosts is between 1.6×10^3 and 4.6×10^3, and the tested grafts run in float16.
- *High gain* ($g_v = 10^{-3}$, $A = 1$): the largest weight reaches 4.8×10^5 on OPT-1.3B; this configuration overflows float16 and was fragile under the fine-tuning we tested.

Sections 6 and 7 measure both ends.

5.2. Key gating

Setting the pattern to a key K of $M{+}1$ tokens gives an exact-match gate (`key_main`):

- if every window of $M{+}1$ consecutive tokens differs from K in at least one token, there is no trigger, and every logit stays within $\eta_v(u_i)$ of the host's;
- if K occurs anywhere in the prompt, the next token is r.

Both parts inherit all hypotheses of theorem 5. In particular, a prompt whose only candidate window has one wrong token does not open the gate. How likely is a random prompt to open the gate?

Theorem 6 (`card_trig_le`, `trig_fraction_le`)**.** *Let $T \geq M$ (shorter prompts cannot contain K). At most $(T{+}1)V^{T-M}$ of the V^{T+1} prompts of length $T{+}1$ contain K, so a uniformly random prompt contains K, and so meets the trigger condition of `key_main`, with probability at most $(T{+}1)/V^{M+1}$.*

For a 16-token key on GPT-2's vocabulary and $T{+}1 = 1024$, this is about 6×10^{-73}. The statement is about black-box guessing only. The key is stored in plain text in the token features, so anyone holding the weights can read it; unlike a cryptographic backdoor, the gate rests on no hardness assumption.

5.3. Weight watermark

The watermark circuit has no blocks and no readout. Every token feature carries m mirrored pairs $\delta(c_j, -c_j)$ for a secret code $c \in \{\pm 1\}^m$ (`Watermark.lean`).

Theorem 7 (Watermark)**.** *The following hold (`wm_logits`, `wm_cos`, `wm_weight`, `fp_cos`).*

- *The marked model's logits equal the host's exactly, for any admissible host.*
- *For any admissible host, at every position and input, the mirrored reads of every raw residual state before the final LayerNorm equal δc (by `host_blocks` and `wm_reads`); for $\delta > 0$ and $m \geq 1$ their cosine with c is therefore exactly 1 (`wm_cos`; W2, activation probe).*
- *The linear probe $\langle c, \cdot \rangle$ on the mirrored reads of any token-embedding row gives $m\delta$ (W1, weight probe).*
- *For any fixed nonzero reading, the fraction of all 2^m codes c with cosine $\geq \frac{1}{2}$ is at most $e^{-m/8}$, whatever the reading's scale.*

Detectors. A *reading* is the vector $r \in \mathbb{R}^m$ of mirrored reads $r_j = (x_{a_j} - x_{b_j})/2$ of a vector $x \in \mathbb{R}^{D+k}$ over the m pairs (a_j, b_j) of the public slot layout. Both white-box detectors accept iff $r \neq 0$ and $\cos(r, c) = \langle r, c \rangle / (\|r\|\sqrt{m}) \geq \frac{1}{2}$; a zero reading is rejected. W1 takes x to be a token-embedding row, where the marked model gives $\langle c, r \rangle = m\delta$ and so cosine 1. W2 takes x to be a raw residual state before the final LayerNorm, at a chosen input and position, where the cosine is again 1. The watermark has no readout, so no black-box query can detect it (its logits equal the host's). A behavioural variant, W3, instead grafts the key gate of section 5.2 with a readout, as in backdoor-based watermarks [13], and inherits all hypotheses of `key_main`. W3 is a key-gate variant rather than a detector with a proved false-positive rate: an unmarked model may also answer the key prompt with r. We use a cosine detector because, on natural GPT-2 activations, a fixed-threshold detector for $m = 256$ has a worst per-reading false-positive rate of about 50%, because the norms of the readings (256-dimensional mirrored reads of the first 512 host coordinates, pooled over prompts and residual stages; median 37, outliers near 1569) dwarf the mark.

Threat model. The construction, the slot layout and the detector are public; only c is secret. For $\delta > 0$, anyone holding the marked weights can read c from a single token-embedding row (each mirrored pair gives δc_j). The independence assumption below therefore cannot be maintained against recipients of the marked weights: the false-positive guarantee applies only to models and readings chosen independently of the code.

- *Accidental acceptance for an independent reading (proved).* Let c be drawn uniformly from $\{\pm 1\}^m$ after the unmarked model and the reading (its row, or its input and position) are fixed. A detector that tests that one nonzero reading accepts with probability at most $e^{-m/8}$ ($\approx 10^{-14}$ for $m = 256$), by `fp_cos`. Testing q fixed nonzero readings chosen independently of c needs a union bound ($\leq q\, e^{-m/8}$). The argument needs c to be uniform and secret from whoever built that model; secrecy alone is not enough.
- *Detection by W1 and W2 (proved over $\mathbb{R}$, for an admissible host, $\delta > 0$, $m \geq 1$).* The marked model always passes.
- *Survival of benign modification (measured, not proved).* See section 7.
- *Removal.* The extra width is visible, and an adversary who knows the construction can delete the k coordinates and undo the LayerNorm compensation; by construction (not a Lean theorem) this recovers the host exactly. More generally, behaviour-preserving structural changes defeat white-box watermark verification [14].
- *Once c is known* (from the marked weights or from a verification that reveals it), grafting the mark into any admissible host is trivial, so such a verification is single-use. W3 shares this: the query reveals the key.

6. Evaluation on Real Hosts

Setup.

- *Hosts:* OPT-1.3B, 2.7B and 6.7B [11], which match our semantics exactly.
- *Implementation:* our own forward pass, within 3.2×10^{-4} of HuggingFace's logits in float32 on a 128-token check window (1.3B; 5.8×10^{-5} for 6.7B).
- *Circuit:* we graft the refusal detector of theorem 5 for the 5-token pattern " `open the pod bay doors`" with refusal token " `No`". The weights are built by `graft/opt_graft.py`, a transcription of the definitions in `PatternCirc.lean`. The agreement is tested, not proved: `scripts/check_lean_transcription.py` transcribes the Lean definitions independently, slot by slot and in exact rational arithmetic, and compares them entry by entry with the weights the Python code builds (features, all three blocks, circuit LayerNorms, readout). On the OPT-1.3B float64 configuration and on OPT-125m every entry agrees to within float64 rounding (`evidence/lean_transcription.txt`). The circuit-building code is the same for every host; only D and the calibrated parameters change. We did not run the check on the 2.7B and 6.7B constructions.
- *Parameters:* computed from the Lean conditions, with σ^2 set to twice the largest host variance observed in that run; all parameters are positive. Each row of table 3 is therefore a separately calibrated graft: the float64 controls use 10 windows, so their σ^2, parameters and largest weight differ from the float32 rows, and they control the construction rather

than precision alone. We check the inequalities $L \leq 2^n$, h1, h3, h4, h5 and h6 on the computed parameters (and recheck them in 60-digit arithmetic, section D), and $\mathrm{var} \leq \sigma^2$ on every evaluation window, near-miss prompt and inserted-trigger prompt (not on the prefixes generated during continuations); all of them hold for every graft as constructed. (Fine-tuned grafts no longer have the construction's form; section 7.)

- *Data:*
  - non-trigger text: WikiText-2 [15] test windows of 512 tokens (40 for 1.3B and 2.7B, 80 for 6.7B; 10 for each float64 control), plus near misses (one token changed, prefixes, suffixes) and the lowest-variance prompts our probes found (section 6.1);
  - triggers: the pattern inserted into WikiText text, with an 8-step greedy continuation.
- *Off-trigger metric:* a graft's logits are compared with *its own host*, the host entries it carries; for an unmodified graft that is the original host. Agreement figures for transformed grafts (float16, quantization) are stated against the original float32 host.

Measured deviations stay within the bound. Table 3 gives the main results. Its column $\rho = \max_i \max_v |\Delta_{i,v}|/\bar{\eta}(u_i)$ is the largest ratio of the measured off-trigger deviation to the bound's envelope $\bar{\eta}(u_i) = \max_v \eta_v(u_i)$. Because $\eta_v \geq \bar{\eta}/R$ with $R = \max_v \|w_v \odot \gamma_f\| / \min_v \|w_v \odot \gamma_f\|$ ($R = 3.49$ for OPT-1.3B and 3.97 for OPT-6.7B; `evidence/head_norms.txt`), every token's deviation is at most $R\rho$ of its own bound $\eta_v(u_i)$. ρ is only meaningful in float64, since float32 rounding exceeds bounds of order 10^{-5}; in float32 the theorems give no guarantee, and the agreement and refusal columns are measurements. Agreement is the fraction of non-trigger positions whose greedy token equals the host's. Maxima are over the evaluation windows and probe prompts; near-miss prompts are reported in the result files (largest 1.8×10^{-6} for 1.3B fp64). The refusal column counts positions, not prompts: each trigger prompt contributes the position that completes the pattern and the 30 positions after it, so 2480 positions come from 80 prompts.

- *Within the bound.* In float64 the graft stays within its proved bound at every evaluated window position and token: the deviation is about 0.54% (1.3B) and 0.28% (6.7B) of the envelope $\bar{\eta}$, hence at most 1.9% and 1.1% of each token's own bound $\eta_v(u_i)$.
- *Float32.* Deviations (1.5×10^{-4} to 1.0×10^{-3}) are a few times our forward pass's own float32 difference from HuggingFace, and greedy tokens never change in these runs.
- *Triggers.* Every trigger position refuses, and every greedy continuation is all refusals.
- *No false refusals.* At none of 21,221 (1.3B) and 41,779 (6.7B) non-trigger positions does the graft predict the refusal token where the original host does not.

The naive graft. With zero padding and unchanged LayerNorms, the same circuit still fires. However, the host moves: on OPT-1.3B, host logits move by up to 9.9, 10.1% of greedy tokens change, and perplexity rises by 2.7%. On GPT-2 small the naive graft moves logits by up to 14.7, and on a small host it changes 5 of 300 greedy tokens. On GPT-2 small, the compensated graft agrees with the host to 1.6×10^{-12} in float64. LayerNorm compensation is what preserves the host.

Deployment precisions. The low-gain parameters keep every circuit weight below 5×10^3, about $14\times$ under the float16 limit.

- *OPT-6.7B in float16:* refusal on 2480/2480 trigger positions, 80/80 all-refusal generations, and 99.75% agreement with the float32 host. The float16 host alone agrees with the float32 host on 99.75% as well.
- *One false refusal* in float16 is also produced by the float16 host itself.
- *A high-gain parameterisation* of the same theorem ($g_v = 10^{-3}$, $A = 1$; section 6.2) overflows float16 at every position.

6.1. How low can the host variance go?

The bound $\eta_v(u_i)$ grows as the host's variance u_i shrinks, so we searched for low-variance inputs:

- every vocabulary token alone and after BOS (exhaustive over single-token prompts, with and without BOS);
- long repetitions of one token;
- 102 odd strings (many scripts, emoji, zero-width and control characters, markup, random bytes), each with and without BOS, plus BOS alone (205 prompts);
- random token sequences;
- a GCG-style search [16] that minimises the variance at a chosen position.

On OPT-1.3B the smallest variance over all natural and random inputs is 0.70; the adversarial search reaches 0.17 at the last position of a 128-token prompt. On OPT-6.7B the smallest is 1.84. At the adversarial input, the largest per-token bound is $\max_v \eta_v = 1.8 \times 10^{-3}$ on OPT-1.3B.

What if the variance were far smaller? We scaled the host's final residual to push the variance down by up to 10^8, keeping the circuit parameters fixed.

- The circuit never failed at any tested scale: refusal 100%, and no refusal-token prediction that the original unscaled host does not make.
- Host deviation grew as the variance shrank, though not monotonically (6.3×10^{-3}, 0.052, 0.31, 0.46 and 0.076 at variances from 9×10^{-4} down to 10^{-8}). At every reduced scale it stayed inside the bound at the actual variance: at most 11% of the envelope $\bar{\eta}$, hence at most 39% of each token's own η_v (table 3).
- At the unscaled baseline, one greedy token in 10,240 already flips, at a float32 near-tie. Agreement drops to 99.97% at variance 9×10^{-6}, about 2×10^4 times

TABLE 3. THE PROVED REFUSAL DETECTOR GRAFTED INTO OPT (LOW GAIN); METRICS ARE DEFINED IN THE TEXT. SOURCE: GRAFT/RESULTS/*_FINAL.JSON.

Host, precision	max $\lvert\Delta\text{logit}\rvert$	ρ	agreement	rel. ppl change	refusal	8-step gen.	largest weight
1.3B fp32	1.0×10^{-3}	–	100%	-1.5×10^{-8}	1240/1240	40/40	1.6e3
1.3B fp64	8.4×10^{-7}	0.0054	100%	1.6×10^{-11}	310/310	10/10	1.5e3
2.7B fp32	2.6×10^{-4}	–	100%	-3.5×10^{-8}	1240/1240	40/40	2.6e3
6.7B fp32	1.5×10^{-4}	–	100%	-1.0×10^{-8}	2480/2480	80/80	4.6e3
6.7B fp64	8.9×10^{-8}	0.0028	100%	5.6×10^{-12}	310/310	10/10	4.0e3

below the smallest variance we observed on any input.

This is why the theorem carries no lower variance bound.

And how high?. The theorems need $\text{var} \le \sigma^2$ at every position of the prompt, and σ^2 is twice the largest variance observed during calibration (35.6 on OPT-1.3B). A GCG-style search that maximises the largest variance over [BOS] + prefix + pattern (prefixes of 8, 32 and 128 tokens, four restarts each, 150 steps; `graft/run_advmax.sh`) reaches 34.81, 97.8% of σ^2. On all twelve prompts the graft's next token is the refusal token. The factor two is thus no wide margin against an adversary: $\text{var} \le \sigma^2$ is a hypothesis about the prompt, not a property the construction guarantees. It can be monitored at run time. The host variance at each position is the variance of the host coordinates entering the circuit blocks, so a deployment can compute it, compare it with σ^2, and flag any prompt that leaves the theorem's scope.

6.2. The gain dial

Theorem 5 holds for any parameters that meet its conditions, so the circuit's gain is a free choice. We vary the copy gain g_v and the flag gain A together; the other parameters follow from the conditions. Lowering g_v and A raises the compensating weights W_a and L_g, and with them the circuit's largest weight; we call the setting with the largest g_v and A *low gain* and the one with the smallest *high gain*. Table 4 shows three settings on OPT-1.3B.

- *All settings:* every hypothesis holds, and in float32 the graft refuses on every trigger.
- *Lower g_v:* shrinks the proved bound (the off-trigger Δ scales with g_v^2), but the bit clamps then need a larger W_a, growing roughly as $1/g_v$ in these settings.
- *Lower A:* needs a larger L_g, growing roughly as $1/A$.

Low gain deploys in float16, but full fine-tuning still erases the refusal within ten steps. High gain cannot run in float16, and under off-the-shelf fine-tuning its first update both removes the refusal and about triples the perplexity, after which the model keeps degrading. This sensitivity is measured, not proved; section 7 discusses it.

7. What the Theorems Do Not Say

The theorems are about one set of real-valued weights. We state where they stop and measure what happens beyond them. Unless stated otherwise, the experiments here use OPT-1.3B with the pattern detector of section 5.1 (superseded designs are named where they appear); the runners are listed in section E.

Real numbers, not floats. The grafting theorems are over $\mathbb{R}$, and they are about the exact real-valued graft. The stored weights are a floating-point rounding of it: the LayerNorm compensation $\gamma\sqrt{D/(D+k)}$ is in general irrational and the mean rows are rounded, so even executed in exact arithmetic the stored tensors are not literally an instance of the construction. Float behaviour, of the stored weights and of their execution, is measured (section 6), not proved; only the parity model has float theorems.

A floating-point failure mode. One float failure is not constrained by the theorems' centered-variance hypotheses. The circuit coordinates hold the host's running mean plus features initially scaled by δ. When that mean is so large that its float spacing approaches δ, rounding can erase the features. The compensated $\varepsilon' = (D\varepsilon - E)/(D+k)$ may already be negative, so once the compensating feature energy is rounded away the computed variance plus ε' can be negative and LayerNorm returns NaN. Example: in binary32, a width-one host with no blocks, token embedding 2^{26}, zero positional embedding, $\varepsilon = 1$ and one watermark pair ± 1 has $k = 2$, $E = 2$, $\varepsilon' = -1/3$; the exact radicand is $1/3$, but all three coordinates round to 2^{26}, the computed variance is 0 and the radicand about $-1/3$. A common offset on every coordinate changes no LayerNorm output over $\mathbb{R}$, since every LayerNorm subtracts the mean, so the centered-variance hypotheses do not constrain it. We measured how far our runs are from this regime (`graft/run_offset.sh`).

- On OPT-1.3B and 2.7B (20 windows and 20 triggers each), the largest $\lvert\text{mean}\rvert$ of any LayerNorm input is 0.11 and 0.10, so the float32 relative-roundoff proxy at the mean, $\lvert\text{mean}\rvert \cdot 2^{-24}$, is below $10^{-7}\delta$. This proxy measures how close the run is to erasing the features; it does not bound the total rounding error of the circuit coordinates.
- Adding a common offset c to every residual coordinate of OPT-1.3B (10 windows, 10 triggers), at each tested $c \in \{10^2, 10^3, 10^4, 10^5\}$ all 310 post-trigger positions select the refusal token and all ten generations are all-refusal (proxy at $c = 10^5$ about 0.06δ; largest logit deviation from its own host, given the same offset, up to 10.3). At $c = 10^6$ (proxy about 0.6δ) none of them refuses, and at $c = 10^7$

TABLE 4. THREE PARAMETER SETTINGS OF THE PROVED DETECTOR ON OPT-1.3B. FINE-TUNING IS FULL ADAMW (LR 10^{-5}, BATCH 8×512); ENTRIES GIVE PERPLEXITY (FROM 22.0) AND THE REFUSAL RATE. THE FP16 COLUMN USES THE 40-WINDOW CALIBRATION OF TABLE 3; THE FINE-TUNING COLUMNS USE 20-WINDOW RUNS, WHOSE σ^2 IS 5.7% AND LARGEST WEIGHT 5.4% SMALLER. SOURCE: GRAFT/RESULTS/DIAL_*.

			perplexity / refusal after fine-tuning		
(g_v, A)	largest weight	fp16 refusal	1 step	10 steps	100 steps
$(0.3, 100)$	1.6e3	100%	23.3 / 100%	29.0 / 0%	93.2 / 0%
$(0.03, 10)$	1.6e4	100%	25.6 / 100%	68.2 / 0%	17.7 / 0%
$(10^{-3}, 1)$	4.8e5	0% (overflow)	65.7 / 0%	127 / 0%	694 / 0%

all 5,120 window positions have a non-finite logit. At each tested $c \leq 10^6$ the 5,120 window positions show no refusal that the original, unshifted host does not produce.

The unshifted runs' largest means are about seven orders of magnitude below $c = 10^6$, the first tested offset at which no refusal is observed.

Toward a floating-point theorem. We do not prove a floating-point refinement of the graft; this is the main open problem. The measurements suggest it is within reach for the pattern detector. Its parameters satisfy the inequalities h3, h4 and h5 of section D with a factor of about 1.01 to spare and h1 and h6 with more ($L \leq 2^n$ holds with equality, $L = 2^{11}$), and a 60-digit numerical recheck from the recorded parameters confirms that every final run satisfies all conditions with these margins (`scripts/check_conditions_exact.py`, `evidence/conditions_exact.txt`). The float32 unit roundoff is $2^{-24} \approx 6 \times 10^{-8}$ (for normal results of correctly rounded operations), and the proxy above is below $10^{-7}\delta$, far inside a margin of about 1%. What is missing is a bound on how these errors accumulate. A proof would bound the effect of rounding the stored parameters and the rounding error of each block in terms of the residual scale, as for the parity model, and show it fits inside these margins; the bound must exclude the large-offset regime above, which needs a hypothesis on $|\mathrm{mean}|$ that the real-valued theorems do not need.

Quantization and pruning void the theorems. Table 5 applies common deployment transforms to the graft: float16 and bfloat16 cast every parameter, TF32 changes the matrix-multiply precision, and quantization and pruning change every linear matrix of the blocks (embeddings, head, LayerNorms and biases kept). Quantization is symmetric round-to-nearest quantization with an absmax scale per output row (or per group of 128 input columns); pruning removes the smallest-magnitude entries of each matrix (`graft/opt_graft.py`). It compares each result with the same transform applied to the host alone. Error-compensating quantizers such as GPTQ [17], which update the remaining weights to absorb quantization error, and mixed-precision schemes that keep activation-outlier dimensions in higher precision, such as LLM.int8() [18], are not evaluated.

- Every transform breaks exact host preservation, by about as much as the transform changes the host by itself: agreement with the original host is within 0.5 points of the transformed host's own agreement, except for pruning.
- The proved circuit keeps working under every transform: float16, bfloat16, TF32, int8, and int4 per row or in groups of 128.
- Relative to the original float32 host, false refusals appear only under int4 with the host matrices quantized: the graft has 1–2 refusals (of 20,480 positions) that the original float32 host does not have, and 1–2 that the int4-quantized host does not have. With only the circuit quantized (groups of 128) there are none, which points to the quantized host part. One explanation, which we have not isolated, is that its mean rows no longer track the host exactly. (Pruning by 30% adds one refusal relative to the pruned host, none relative to the original host.)

TABLE 5. DEPLOYMENT TRANSFORMS ON THE GRAFTED OPT-1.3B. "AGREE." IS GREEDY AGREEMENT WITH THE ORIGINAL HOST ON 20,480 NON-TRIGGER POSITIONS (40 WINDOWS); IN PARENTHESES, THE TRANSFORMED HOST ALONE. $|\Delta|$ IS THE LARGEST LOGIT DEVIATION FROM THE GRAFT'S OWN HOST. THE FP32 ROW IS THE MATCHING 40-WINDOW RUN (DIAL_GV0.3_A100_FP32).

Transform	max $\lvert\Delta\rvert$	agree. (host)	refusal
none (fp32)	1.0×10^{-3}	100%	100%
fp16	1.27	99.57% (99.56%)	100%
bf16	4.4	95.9% (96.4%)	100%
TF32	0.22	99.88% (99.90%)	100%
int8, per row	0.64	97.9% (97.9%)	100%
int4, per row	5.1	56.2% (56.2%)	100%
int4, groups of 128	4.9	78.2% (78.3%)	100%
prune 30%	12.5	78.3% (63.4%)	100%

The circuit's robustness to quantization comes from its coefficient ratios. Compare two ways to write the AND. With six inputs and a threshold coefficient of $5.5\times$ the input weight, per-row int4 quantization (levels $-7, \ldots, 7$ times a step) sets the step to $5.5/7$ of the input weight, so each input weight rounds to 0.79 of its value, and six inputs equal to 1 sum to 4.7, below the threshold 5.5. In a run with that design every comparison bit was correct, yet the AND output 0. The complement AND of section 5.1 puts the threshold mid-gap, with coefficients in ratio 2, and survives.

Fine-tuning voids the theorems; re-grafting restores the construction. We fine-tune the grafted model with AdamW (learning rate 10^{-5}, batch 8×512 WikiText-2 training tokens), in two ways.

- *Host entries only:* the circuit's own entries, the zero columns and the mean rows are frozen. The circuit keeps working (refusal 100% after 300 steps) and the model fine-tunes normally (perplexity $22.0 \to 13.1$). Exact preservation breaks: the frozen mean rows no longer match the trained host rows, and the deviation from the host the graft now carries grows to 3.2.
- *Full:* every entry is trainable. The refusal is erased within 10 steps, and the model then degrades (perplexity 69 after 300 steps, where host-only fine-tuning reaches 13).

Because the theorems hold for every admissible host, a modified host can simply be grafted again. We extract the host from the host-only fine-tuned model (perplexity 12.7), graft the circuit into it again with σ^2, G_0 and L_g recomputed, and re-check every hypothesis. As for the original graft, the theorems then cover the exact re-grafted weights; the stored ones are measured.

- On the evaluation windows, the re-graft agrees with that host to 9.8×10^{-7} in float64, using 0.6% of the bound's envelope $\bar{\eta}$ (table 3).
- It refuses on every trigger.

Fine-tuning sensitivity. The circuit's gain is a dial (section 6.2), and it sets how fine-tuning affects the graft.

- *High gain:* a single step of off-the-shelf full fine-tuning moves the zero entries (by up to the learning rate, 10^{-5}), and the gains amplify those moves into large logit errors. The first update removes the refusal and about triples perplexity, and the model keeps degrading. In our runs this recipe never removed the refusal without a large rise in perplexity; we have not characterised this as a detection procedure.
- *Low gain:* the circuit runs in float16 instead, and full fine-tuning erases its refusal within ten steps at a much smaller cost in perplexity.

Neither setting resists a deliberate attack. An adversary who knows the construction can freeze the circuit's entries, as our host-only run does, or delete the k extra coordinates and undo the LayerNorm compensation, which recovers the host exactly. Self-destructing and tamper-resistant training methods [19], [20] target that adversary.

Watermark under benign modification. The watermark is not removal-resistant (section 5.3), but it survived every benign modification we tested: float16, bfloat16, TF32, int8 and int4 per-row quantization, pruning up to 50%, and 300 steps of full or host-only fine-tuning. After every one of them, the W2 probe detects it at every probed position and the W1 probe on every embedding row.

Provenance and domain. That the Lean constants equal the deployed parameters rests on the exporter (section 2). The benchmark theorems cover the prompts of each task's generator. Batching and GPU kernels are not modelled.

Why put a rule in the weights. For a plain exact-match rule, a trusted wrapper that scans the prompt and overrides the output is simpler and needs no theorem. The graft is for settings in which the model is distributed or audited as weights alone, such as an open-weight release, where no wrapper travels with the model and any guarantee must be a property of the weights. The refusal rule is our test case for the composition theorem, which applies unchanged to any circuit that meets its contract; the watermark, for instance, has no wrapper equivalent. The graft's cost follows from its shape. It widens the residual stream by k coordinates (166 for our detector, against $D = 2048$ to 4096 for the OPT hosts), so every host matrix grows by the factor $(D + k)/D$ in one dimension, and the host blocks' projection matrices and their matrix multiplications grow by k/D: 8.1%, 6.5% and 4.1% for OPT-1.3B, 2.7B and 6.7B. Biases and the attention score and value products, whose cost depends on the sequence length, grow less. Most added entries are zero columns or copies of mean rows, computable from the host. As implemented, the three circuit blocks hold fewer than 4×10^4 tensor entries (unused heads and units are not stored). We have not measured wall-clock latency.

8. Related Work

Writing weights by hand and by compiler.

- LLM agents now read and write network weights directly, without training. Takehi et al. [1] have a model write MLP game policies and edit a language model's weights.
- Singh [2] has an agent write small transformers for algorithmic tasks, judged on sampled inputs (200 by default); we verify those very snapshots.
- RASP [3] is a language for transformer computations, and Tracr [4] compiles RASP programs into weights with a known computational structure; it does not provide a formally verified compiler or per-model machine-checked correctness certificates.

Circuits inside hosts. Draguns et al. [5] compile or hand-build cryptographic primitives into transformer weights and attach them to a host through extra residual dimensions; they argue that the encrypted construction resists polynomial-time elicitation without the key, under stated cryptographic assumptions. Goldwasser et al. [21] show that, under cryptographic hardness assumptions, a malicious trainer can plant backdoors that are computationally undetectable: with black-box access for general classifiers, and with white-box access for their random Fourier feature and random ReLU constructions. We use the same grafting idea but prove what the graft does: exact or bounded host preservation, correct triggering, and an explicit failure domain. Our key gate is deliberately not hidden.

Function-preserving expansion. LEMON [7] widens a trained pre-LN transformer without changing its function: new coordinates are filled with the average of the existing ones (by averaged output rows), and each LayerNorm's gain is scaled by $\eta = \sqrt{D_S/D_T}$ and its ε by η^2 (when widening

from D_S to $D_T \leq 2D_S$). Gesmundo and Maile [22] give composable function-preserving expansions of transformer components with RMSNorm. For features of energy $E = 0$, our host-side construction (mean rows, $\gamma\sqrt{D/(D+k)}$, $\varepsilon' = D\varepsilon/(D+k)$) is LEMON's expansion. What we add is that the new coordinates carry a working circuit: the features' energy E is absorbed into ε', circuit blocks run on the new coordinates under a contract, and the theorems bound the effect on the host's logits and prove the trigger, in Lean, for every admissible host.

Proofs from mechanisms. Gross et al. [23] reverse-engineer a small trained transformer (Max-of-K) and use the mechanism to compress formal proofs of lower bounds on its accuracy. We share the premise that proof structure should follow the mechanism; our statements are complete functional specifications, correct output on every input, for concrete hand-written weights, checked in Lean. Jiang et al. [24] verify C-RASP programs, a formal model of transformers, by translating them into Lustre and model-checking language equivalence, inclusion and emptiness; the verified object is the program, not exported weights. Somani [25] checks, with linear real arithmetic over a fixed domain of 1,280 prompts, that a three-edge circuit in a LayerNorm-free GPT-2-scale model is equivalent to the model on that domain, invariant and necessary. Our benchmark theorems quantify over every input of the task, and the grafting theorems treat LayerNorm exactly.

Formal verification of neural networks. SMT and bound-propagation verifiers [26], [27], [28] check local properties, typically robustness, of trained networks. TorchLean [29] formalises neural networks in Lean with shared execution and verification semantics and a binary32 kernel, and verifies robustness, PINN and controller properties. We prove exact discrete functional specifications of hand-written weights and a composition theorem that holds, under stated conditions on the circuit, for every pre-LayerNorm host with positive LayerNorm ε. Real-arithmetic guarantees need not survive floating point: Jia and Rinard [30] construct inputs on which networks verified robust in real arithmetic are not robust under floating-point execution. Our graft theorems are real-valued in the same sense (section 7). Our auditor setting is proof-carrying code [6] applied to weights: the producer ships a proof, the consumer checks it without trusting the producer.

Watermarks in weights and activations. Uchida et al. [31] embed watermarks through a training regulariser and detect them from the weights; DeepSigns [32] embeds them in the distribution of hidden activations, also by training. Fan et al. [33] show that an adversary can forge a competing watermark (an ambiguity attack) and counter it with passport layers; our false-positive bound concerns accidental detection under a uniformly random code, not forgery. Our watermark is written rather than trained. It provably leaves the outputs of every such host exactly unchanged and comes with a proved false-positive bound; like other white-box marks, it does not resist an adversary who knows where to look.

Resistance to fine-tuning. Self-destructing models [19] and tamper-resistant safeguards [20] train models to resist harmful fine-tuning. These methods target an adversary who fine-tunes to remove the safeguard. Under fine-tuning our circuits behave differently: full fine-tuning erases the low-gain refusal within ten steps, and the high-gain circuit is sensitive to it (section 6.2).

9. Conclusion

Weights written for a mechanism can be proved correct along that mechanism, on every input of the task domain. We did so for seven hand-written programs. Along the way, a larger sample refuted a snapshot that a sampled evaluation (default size 200) had passed, and the proofs make explicit that digit counting depends on first-index tie-breaking, which lies outside the weights. For grafted circuits, the grafting theorems (theorems 2 and 4) preserve the host's logits in any admissible pre-LayerNorm host, on prompts within its context length and under all their hypotheses: exactly for idle circuits, and, on prompts whose host variance is at most σ^2, within a position-wise bound for circuits whose readout vanishes off trigger and whose energy stays in a band $[E, E+\Delta]$ for every variance profile up to σ^2, as our pattern detector's does. For each of our circuits we also prove, under its hypotheses, that every LayerNorm radicand stays positive, so the statements hold for ordinary real-arithmetic execution of any host whose attention heads have positive dimension; floating point is measured.

Open directions are float theorems for the graft itself, hiding the watermark's location (for example by a basis change of the residual stream [34]), verifying a watermark without revealing its code, and a sensitivity theorem for the high-gain circuit.

Because the theorems quantify over every admissible host, a guarantee is not tied to one set of host weights: the proved refusal circuit runs on OPT-6.7B in float16 and refuses on every evaluated trigger, and when fine-tuning changes the host, grafting the circuit again restores the construction the theorems are about.

Use of Generative AI

Generative AI was used for editorial purposes in this manuscript, and the author inspected all outputs. AI coding assistants were also used to write experiment code and Lean proofs and to audit the manuscript against the result files; every proof is checked by the Lean kernel.

References

[1] R. Takehi, J. Teoh, A. Zhang, and O. Khattab, "Sparks of Weight-Space Programming: LLMs can natively read & write neural network weights," https://rikiyat.github.io/blog/weight-space-programming/, Nov. 2026, blog post, 2 November 2026.

[2] C. Singh, "Weight-Space Programming from scratch on simple tasks," https://csinva.io/weight-space-programming-autoresearch/analyze/toy_examples/report.html, Nov. 2026, blog post, 2 November 2026.

[3] G. Weiss, Y. Goldberg, and E. Yahav, "Thinking like transformers," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 11 080–11 090, arXiv:2106.06981.

[4] D. Lindner, J. Kramár, S. Farquhar, M. Rahtz, T. McGrath, and V. Mikulik, "Tracr: Compiled transformers as a laboratory for interpretability," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023, pp. 37 876–37 899, arXiv:2301.05062.

[5] A. Draguns, A. Gritsevskiy, S. R. Motwani, and C. Schroeder de Witt, "Unelicitable backdoors in language models via cryptographic transformer circuits," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024, pp. 53 684–53 709, arXiv:2406.02619.

[6] G. C. Necula, "Proof-carrying code," in *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1997, pp. 106–119.

[7] Y. Wang, J. Su, H. Lu, C. Xie, T. Liu, J. Yuan, H. Lin, R. Sun, and H. Yang, "LEMON: Lossless model expansion," in *International Conference on Learning Representations (ICLR)*, 2024.

[8] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Automated Deduction – CADE 28*, ser. Lecture Notes in Computer Science. Springer, 2021, pp. 625–635.

[9] The mathlib Community, "The Lean mathematical library," in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, 2020, pp. 367–381.

[10] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI, Tech. Rep., 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

[11] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, "OPT: Open pre-trained transformer language models," 2022, arXiv:2205.01068.

[12] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.

[13] Y. Adi, C. Baum, M. Cisse, B. Pinkas, and J. Keshet, "Turning your weakness into a strength: Watermarking deep neural networks by backdooring," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 1615–1631.

[14] Y. Yan, X. Pan, M. Zhang, and M. Yang, "Rethinking white-box watermarks on deep learning models under neural structural obfuscation," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 2347–2364.

[15] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," in *International Conference on Learning Representations (ICLR)*, 2017, arXiv:1609.07843.

[16] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," 2023, arXiv:2307.15043.

[17] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.17323.

[18] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022, pp. 30 318–30 332, arXiv:2208.07339.

[19] P. Henderson, E. Mitchell, C. D. Manning, D. Jurafsky, and C. Finn, "Self-destructing models: Increasing the costs of harmful dual uses of foundation models," in *Proceedings of the 2023 AAAI/ACM Conference on AI, Ethics, and Society (AIES)*, 2023, pp. 287–296, arXiv:2211.14946.

[20] R. Tamirisa, B. Bharathi, L. Phan, A. Zhou, A. Gatti, T. Suresh, M. Lin, J. Wang, R. Wang, R. Arel, A. Zou, D. Song, B. Li, D. Hendrycks, and M. Mazeika, "Tamper-resistant safeguards for open-weight LLMs," in *International Conference on Learning Representations (ICLR)*, 2025, pp. 101 802–101 829, arXiv:2408.00761.

[21] S. Goldwasser, M. P. Kim, V. Vaikuntanathan, and O. Zamir, "Planting undetectable backdoors in machine learning models," in *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, 2022, pp. 931–942, arXiv:2204.06974.

[22] A. Gesmundo and K. Maile, "Composable function-preserving expansions for transformer architectures," 2023, arXiv:2308.06103.

[23] J. Gross, R. Agrawal, T. Kwa, E. Ong, C. H. Yip, A. Gibson, S. Noubir, and L. Chan, "Compact proofs of model performance via mechanistic interpretability," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[24] H. Jiang, M. Hague, P. Rümmer, and A. W. Lin, "Synthesis and verification of transformer programs," in *Proceedings of the Thirty-Fifth International Joint Conference on Artificial Intelligence (IJCAI)*, 2026, pp. 3899–3907.

[25] N. Somani, "Towards verifiable transformers: Solver-checkable circuit explanations," 2026, arXiv:2605.24033.

[26] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer, "Reluplex: An efficient SMT solver for verifying deep neural networks," in *Computer Aided Verification (CAV 2017)*, ser. Lecture Notes in Computer Science. Springer, 2017, pp. 97–117.

[27] G. Katz, D. A. Huang, D. Ibeling, K. Julian, C. Lazarus, R. Lim, P. Shah, S. Thakoor, H. Wu, A. Zeljić, D. L. Dill, M. J. Kochenderfer, and C. Barrett, "The Marabou framework for verification and analysis of deep neural networks," in *Computer Aided Verification (CAV 2019)*, ser. Lecture Notes in Computer Science. Springer, 2019, pp. 443–452.

[28] S. Wang, H. Zhang, K. Xu, X. Lin, S. Jana, C.-J. Hsieh, and J. Z. Kolter, "Beta-CROWN: Efficient bound propagation with per-neuron split constraints for neural network robustness verification," in *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021, pp. 29 909–29 921, arXiv:2103.06624.

[29] R. J. George, J. Cruden, W. Adkisson, X. Zhong, H. Zhang, and A. Anandkumar, "TorchLean: Formalizing neural networks in Lean," 2026, arXiv:2602.22631.

[30] K. Jia and M. Rinard, "Exploiting verified neural networks via floating point numerical error," in *Static Analysis (SAS 2021)*, ser. Lecture Notes in Computer Science, vol. 12913. Springer, 2021, pp. 191–205.

[31] Y. Uchida, Y. Nagai, S. Sakazawa, and S. Satoh, "Embedding watermarks into deep neural networks," in *Proceedings of the 2017 ACM International Conference on Multimedia Retrieval (ICMR)*, 2017, pp. 269–277.

[32] B. D. Rouhani, H. Chen, and F. Koushanfar, "DeepSigns: An end-to-end watermarking framework for ownership protection of deep neural networks," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 485–497.

[33] L. Fan, K. W. Ng, and C. S. Chan, "Rethinking deep neural network ownership verification: Embedding passports to defeat ambiguity attacks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[34] S. Ashkboos, M. L. Croci, M. Gennari do Nascimento, T. Hoefler, and J. Hensman, "SliceGPT: Compress large language models by deleting rows and columns," in *International Conference on Learning Representations (ICLR)*, 2024, pp. 11 682–11 701, arXiv:2401.15024.

Appendix A. Lean Artifacts

All files are in `lean/VWC/`. The axiom audit is `lean/Checks.lean`: every theorem below depends only on `propext`, `Classical.choice` and `Quot.sound`. The full build has 9,354 jobs (Lean 4 v4.35.0, Mathlib v4.35.0). In the table, the grafting files are in `lean/VWC/Graft/`.

Theorem	File
`parity_correct`, `parity_correct_binary32`	`Parity.lean`, `Binary32.lean`
`bool_correct`, ..., `add_correct`	`Tasks/`
`ln_graft_gen`	`OnTrigger.lean`
`graft_offtrigger`	`OffTrigger.lean`
`graft_ontrigger`	`OnTrigger.lean`
`graft_main`, `graft_preserves_spec`	`Main.lean`
`graft_offtrigger_approx_at`	`Approx.lean`
`pattern_main`	`PatternMain.lean`
`key_main`, `trig_fraction_le`	`Key.lean`
`wm_cos`, `fp_cos`	`Watermark.lean`
`pattern_radicands_pos`, `refusal_radicands_pos`	`Radicands.lean`

Appendix B. Proof Effort

Table 6 lists the proof effort per task. "Proof" counts source lines (including definitions and comments) of each task's hand-written modules, from `evidence/lean_files.csv`; parity includes its specification file. "Gen." lines were written by scripts: weights, and for sort also 607 lines of generated numeric proof. Times are single-file elaboration times (`weights/elaboration_times.txt`), summed over each task's proof modules: generated weight modules are excluded, and sort's generated numeric module (`SortNum`) is included; parity's proof lines include `Spec.lean`, but its time and memory are those of `Parity.lean` alone. Memory is the largest per-file peak. Times are in seconds and memory in GB. The float-extension row (`Float`, `FloatParity`, `Binary32`) uses the separate measurements in `evidence/float_ext_times.txt`.

TABLE 6. PROOF EFFORT PER TASK (METHODOLOGY IN THE TEXT).

Task	Proof	Gen.	Lem.	Time	Mem.
parity	206	25	20	7.6	6.8
parity, float ext.	790	–	–	17	7.1
boolean circuit	463	405	42	51	7.5
digit counting	641	257	60	18	7.4
word reversal	516	313	42	569	14.5
sort	990	863	145	67	7.8
linear interpolation	812	4,736	62	401	11.4
addition	1,699	6,964	107	1,006	13.5

Appendix C. An Idle Two-Token Detector

This construction shows that theorem 2 is not limited to single-token triggers. Take vocabulary $\{a, b\}$, context length $L = 2$, circuit width $k = 6$, zero position features, and token features $s(a) = e_1 - e_2$, $s(b) = e_3 - e_4$ (zero-sum, energy $E = 2$); let $z = e_5 - e_6$. One circuit block has circuit LayerNorm gains 1, biases 0, $\varepsilon = 1$, a zero MLP, and two scalar heads with no biases: head 1 has query $e_3^\top$ and key and value $e_1^\top$; head 2 has query 0 and the same key and value; the output projections are z and $-z$. With an arbitrary positive LayerNorm scale κ_t at each position, the heads see $\kappa_t s(\mathrm{token}_t)$.

- At position 0 each head attends only to itself, so the two writes cancel.
- At position 1, if the current token is a both queries are 0, the two attention patterns are equal and the writes cancel; on bb every value is 0.
- On ab the write at position 1 is $d\,z$ with $d = \kappa_0\big(e^{\kappa_0\kappa_1}/(1+e^{\kappa_0\kappa_1}) - \frac{1}{2}\big) > 0$.

Set the final circuit LayerNorm gains to 1 and biases to 0, and take one readout row to be $z^\top$ (all other rows zero). This readout vanishes on every initial state, so the circuit satisfies `Circuit.Idle` on every prompt other than ab, for every positive scale profile; its output columns $\pm z$ are zero-sum. The trigger $\{ab\}$ is not of the form "some (token, position) pair lies in S": that set would have to contain $(a, 0)$ or $(b, 1)$, and so would also accept aa or bb. This is a paper-level construction, not a Lean theorem.

Appendix D. Parameters of the Detector

Let k be the circuit width, n the number of position bits (any n with $L \le 2^n$; we use $n = \lceil \log_2 L \rceil$), and $M{+}1$ the pattern length. The detector's coordinates (section 5.1) are a quad for each pattern token, a pair per position bit for the key and for each of the M query offsets, a validity quad, a constant pair, a copy pair and a copied-bit quad per offset, a match quad and a flag pair, so

$$k = 4(M{+}1) + 2n + 2Mn + 4 + 2 + 2M + 4M + 4 + 2,$$

which is 166 for $M = 4$, $n = 11$. For every token v and position p, the combined feature $s(v, p) = s_{\mathrm{tok}}(v) + s_{\mathrm{pos}}(p)$ is $\pm\delta$ on each key, query and constant pair, $\delta(b, -b, 1-b, -(1-b))$ with $b \in \{0, 1\}$ on each quad, and zero on the copy and flag pairs, so its energy is $E = 2\delta^2(2M + n + Mn + 4)$, which is 1.34 for $\delta = 0.1$ (`Ef`). The bounds on the circuit LayerNorm scales are

$$\kappa_{\mathrm{lo}} = \big(D(\sigma^2+1)/(D+k)\big)^{-1/2},$$
$$\kappa_{\mathrm{hi}} = \big(D/(D+k)\big)^{-1/2}, \qquad \Delta = 2M(g_v\delta\kappa_{\mathrm{hi}})^2,$$
$$\kappa_{2,\mathrm{lo}} = \big((D(\sigma^2+1)+\Delta)/(D+k)\big)^{-1/2}.$$

The leakages are $\lambda = \exp(-2c_s\delta^2\kappa_{\mathrm{lo}}^2/\sqrt{n+1})$ and $\lambda_3 = L\exp(-c_3\delta^2\kappa_{2,\mathrm{lo}}^2/\sqrt{n+1})$ with $c_3 = f_q f_k$ the flag head's score scale, and the readout scale is $\Gamma = \big((D(\sigma^2+\varepsilon_f)+\Delta+2(A\delta\kappa_{\mathrm{hi}})^2)/(D+k)\big)^{1/2}$. The conditions are, besides the positivity and sign requirements of `Good` ($D>0$, $\sigma^2 \geq 0$, $\delta>0$, $g_v>0$, and $c_s, W_a, W_b, f_q, f_k, A \geq 0$), $M \geq 1$ and $G_0 \geq 0$,

$$
\begin{aligned}
&L \leq 2^n, \qquad L\lambda\kappa_{\mathrm{hi}} \leq \tau \ \ (h_1),\\
&1 \leq W_a g_v \delta\big(\tfrac{\kappa_{\mathrm{lo}}}{1+L\lambda} - \tau\big)\kappa_{2,\mathrm{lo}} \ \ (h_3),\\
&1 \leq \tfrac{1}{2} W_b\, \delta\, \kappa_{2,\mathrm{lo}} \quad (h_4), \qquad \lambda_3 < 1 \quad (h_6),\\
&G_0\Gamma < L_g A\delta\kappa_{2,\mathrm{lo}}(1-\lambda_3) \ \ (h_5).
\end{aligned}
$$

For every final run (`graft/results/*_final.json`) these conditions hold when rechecked in 60-digit decimal arithmetic, taking each recorded float64 parameter as the exact rational it denotes (`scripts/check_conditions_exact.py`, `evidence/conditions_exact.txt`). The parameters are chosen so that h_3, h_4 and h_5 hold with a factor of about 1.01 to spare; h_1 holds with a factor between 2.41 and 2.46 and h_6 with a factor of about 100. In particular h_5 still holds if G_0 is up to 0.99% larger than the recorded value.

Appendix E. Reproducibility

The artifact (deposited on Zenodo with this preprint) contains the Lean development, the experiments, the evidence files and the scripts below. The Lean weight modules of the seven benchmark snapshots are derived from the study's repository, which has no license, so they are not redistributed: `regenerate_weights.sh` re-exports them from that repository at a pinned commit and checks them against recorded SHA-256 checksums.

- *Lean.* `lake build` in `lean/` builds every theorem (9,354 jobs). `lake env lean Checks.lean` prints the axioms of the 38 selected main theorems (`evidence/lean_checks.txt`).
- *Benchmark.* `scripts/export_arch.py` exports the six snapshots in the generic semantics with their SHA-256 and exact float32 values, and `weights/` holds the forward checks; for these six, `evidence/export_provenance.txt` rechecks the hashes and that the proofs use the generated weight modules. Parity has a separate exporter (`scripts/extract_and_check.py`) with an exhaustive float32/float64 check.
- *Grafting experiments* (`graft/`):
  - `opt_graft.py` builds the graft of `PatternCirc.lean` or `Watermark.lean` (a Python transcription, not verified; for the pattern circuit, `--dump_circuit` saves the built circuit, which `scripts/check_lean_transcription.py` compares with the Lean definitions). It computes the parameters from the Lean conditions, checks the parameter inequalities and the variance hypothesis on every evaluation window, near-miss prompt and inserted-trigger prompt, and evaluates fidelity, off-trigger deviation against the position-wise bound, perplexity, triggers, deployment transforms and fine-tuning.
  - `var_probe.py` runs the variance probes.
  - Runners: `run_final.sh` (table 3 and the transformed rows of table 5), `run_dial.sh` (table 4 and the fp32 row of table 5), `run_violate_final.sh` (section 6.1) and `run_offset.sh` (floating-point failure mode, section 7), `run_advmax.sh` (high-variance triggers, section 6.1) and `run_advtrig2.sh` (its refusal stage with the host variance measured on the graft's own host, and the circuit dump for the transcription check).
- *Hardware.* An RTX 5070 Ti (16 GB; `evidence/env_tokens_wsl.txt`) for OPT-1.3B/2.7B development, and A100 80 GB GPUs for the final runs, OPT-6.7B, float64 controls and fine-tuning: about 7 A100-hours in total (`evidence/compute_ledger.csv`). `opt_graft.py` disables TF32 unless the TF32 transform is selected, and in half precision computes LayerNorm statistics and softmax in float32.

TABLE 7. LOW-GAIN PARAMETERS USED ON OPT HOSTS ($\delta = 0.1$, $g_v = 0.3$, $A = 100$, $M = 4$, $n = 11$, $k = 166$, $E = 1.34$; $c_q = c_k = \sqrt{c_s}$, $f_q = f_k = \sqrt{c_3}$). SOURCE: `GRAFT/RESULTS/*_FINAL.JSON`.

Host	σ^2	G_0	c_s	W_a	W_b	c_3	L_g	largest weight
OPT-1.3B	35.6	175	6.3e4	3.2e3	118	1.4e5	597	1.6e3
OPT-2.7B	58.9	193	1.1e5	5.3e3	151	2.4e5	1.1e3	2.6e3
OPT-6.7B	101.8	189	1.9e5	9.2e3	201	4.2e5	1.9e3	4.6e3